

GDN Tree-Scan: Correct State Commitment and Performance Limits of Tree Speculation in Recurrent Hybrids

Zhiyuan Ma

zhiyuanma0520@gmail.com

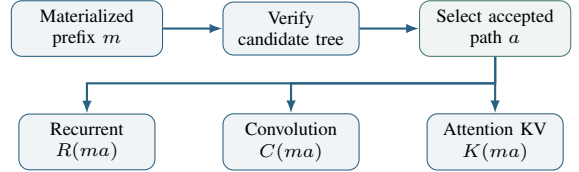
Abstract—Tree speculative decoding for recurrent-hybrid language models must preserve both candidate ancestry and the state of the selected continuation. We present a systems study of GDN Tree-Scan, a Gated DeltaNet verifier integrated with vLLM and FlashAttention-2. The implementation coordinates branch-local recurrent computation, convolution history, attention key–value remapping, and device-side selection. Archived regression evidence identifies two distinct failures: incomplete accepted-path state publication and a physical-layout change that perturbs the shared attention spine despite masking. The publication repair changes a token fixture from 15 failures in 15 checks to zero; the layout repair produces byte agreement on the shared spine in its tested route. Performance does not follow acceptance alone. In a historical batch-four-configured campaign on Qwen3.6-27B-FP8 and GB10, a tree commits 5.286 tokens per request verification event against a five-step native chain’s 4.422, but its complete decode-step rate is 32.85 versus 42.74 tokens/s. A separate paired attention optimization reduces its tested step by 27.03 milliseconds. A kernel-level pilot on identical operands finds that the sequential scan, a forward-substitution (WY-family) solver, and a finite-Neumann (Bole-family) solver all meet a float64 oracle at the fp32 floor when dense products run in fp32, that tf32 products cost three orders of magnitude, and that the production scan reproduces the native verify kernel bit-for-bit on the tested operands in an fp32-store diagnostic; it does not test model-level continuation or fresh prefixes. These findings are scoped to archived configurations, with effective sampling provenance and a matched final-stack comparison still pending. The study provides concrete state and numerical witnesses and reconciled cost accounting, rather than a general superiority claim or a deployed distribution-equivalence proof.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

This paper studies that boundary in GDN Tree-Scan, a Gated DeltaNet verifier integrated with vLLM and a fork



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 1. A selected token path must have a matching continuation state. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled token can remain pending until the next forward, just as in native decoding.

of FlashAttention-2. The implementation uses branch-local recurrent computation, device-side multidraft selection, and accepted-path publication across recurrent state, convolution history, and attention key–value (KV) storage [6], [7]. An attention-KV mapping error demonstrates why repairing only the recurrent state is insufficient: a valid selected token path can still read a rejected sibling on the next iteration. A second failure arises even when masked branches contribute zero attention weight. Moving the visible spine keys changes floating-point reduction association and can alter a near-tied verification decision [8].

Acceptance length also leaves the performance question incomplete. In one archived campaign configured for batch four, a tree committed 5.286 tokens per request verification event against native five-step multi-token prediction’s 4.422. Its event-normalized complete decode cost increased from 103.45 to 160.88 ms, and the corresponding decode-step rate fell from 42.74 to 32.85 tokens/s [9], [10]. This result describes that implementation and configuration. It neither establishes an impossibility result for trees nor compares the current code. It motivates explicit accounting for drafting, verification, commitment, host work, and serving-memory pressure.

The revision makes three bounded contributions. First, it states the accepted-prefix contract at the actual serving boundary and associates it with concrete regression witnesses. Second, it explains a physical-layout source of numerical spine disagreement and the tested repair. Third, it reconciles archived acceptance and timing results, including a strong depth-matched chain control and a later paired attention-kernel improvement.

The latter reduced its tested batch-four step time by 27.03 ms, but does not establish tree-over-chain superiority [11].

Parallel GDN tree verification and compact accepted-state reconstruction are already addressed by direct related work. Section II positions this study accordingly. We do not claim a first hybrid-tree system, general bitwise equivalence, or a measured advantage over those implementations. Section VII separates the experiments needed for stronger current-system claims from the historical findings already available.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [12]–[14]. Our study keeps these algorithmic questions separate from whether a deployed kernel and cache return the intended probabilities.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [15], [16]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [17], [18]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. FlashAttention-2’s work partitioning supplies the attention substrate examined here [19].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [20]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [21], [22]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It stores compact update factors and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and online replay of agent sessions [23]. Its state and serving mechanisms directly overlap the problem studied here. Our measured replay cost is therefore not a lower bound on alternative implementations.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [23].

TABLE I
DIRECT COMPARISON SCOPE. EXTERNAL METHODS WERE READ, NOT EXECUTED IN THIS REVISION.

Method	Overlapping mechanism	Boundary for this study
Bole [23]	Parallel verifier; factorized commit	Different stack and precision; no matched run
TreeWY [24]	Tree solve; accepted-state reconstruction	Different models and operating regime; no matched run
ReplaySSM [25]	Input caching; deferred state work	Alternative state policy; no matched run
GDN Tree-Scan	Scan/replay; accepted-path remapping	Specific numerical and lifecycle witnesses; historical cost evidence

Our path preserves sequential updates and replays accepted operands; Section IV-D explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. Its vLLM evaluation uses Qwen3.5-35B and 397B on B200, greedy verification, and disabled prefix caching. It explicitly permits finite-precision disagreement with the baseline and reports that wider trees improve acceptance without yet winning throughput [24]. Consequently, shared-spine byte checks in our particular route neither establish that TreeWY is incorrect nor replace its serving comparison.

ReplaySSM caches recurrent inputs and changes when state is materialized; its implementation RFC includes GDN and Mamba2 [25]. Reconstruction and replay are not claimed as new concepts here. Table I states the comparison boundary. Published speed ratios from different checkpoints, precision formats, workloads, and software builds are not combined into a performance ranking.

III. THE ACCEPTED-PREFIX CONTRACT

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(S(m), b)$, not a claim that $S(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 1 illustrates the three state surfaces documented by the implementation audit [7], [10].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model's computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. Four different correctness questions

Algebraic correctness asks whether the recurrence or its reformulation computes the same real-arithmetic function. *Numerical agreement* asks whether particular kernels, layouts, and precisions agree with a selected reference. *Sampler correctness* asks whether the acceptance/residual rule returns its supplied target distribution. *Serving correctness* asks whether these properties survive cache restoration, row reuse, batching, and the next iteration [2], [10], [20].

For example, for target probabilities p and proposal probabilities q , accepted mass is $\min(p, q)$ under ordinary rejection sampling. Sampling the remaining mass proportional to $(p - q)_+$ gives the target marginal. The argument is conditional on the actual proposal law and target probabilities; substituting stale logits or an incorrect state changes the distribution being sampled [1], [2]. The same distinction applies to the multidraft reference used here [3], [6].

If every iteration supplies the reference conditional probabilities and publishes an equivalent continuation boundary, an induction over emitted tokens gives the reference joint distribution. This is a conditional argument, not a proof that the implemented system satisfies its premises. A tolerance test, zero observed garble, or equal task outcomes does not discharge all those premises.

IV. IMPLEMENTATION AND FAILURE MECHANISMS

A. A descriptor shared by verification and commitment

Figure 2 shows the served path. Native multi-token prediction (MTP) supplies the principal draft chain; selected variants add runner-up candidates. The descriptor maps each candidate through the attention mask, recurrent scan, sampler, and selected-path publication. Historical variants also use a heuristic

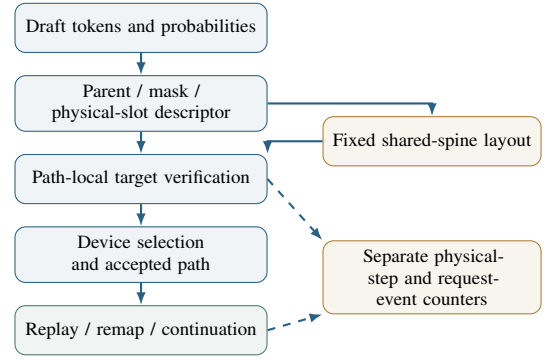


Fig. 2. One descriptor connects the target forward and the continuation state. The selected row also controls the next drafter input. Timing records must identify whether they count a physical step or a request verification event.

Algorithm 1 Logical verifier and continuation boundary

Require: Reference-aligned state boundary, candidate descriptor T

- 1: Map logical candidates to physical slots and positions.
- 2: Verify using path-local convolution, GDN, and attention.
- 3: Apply the configured sampler to matching target/proposal rows.
- 4: Obtain the accepted path and any pending correction token.
- 5: Publish recurrent and convolution state for the materialized path.
- 6: Map its attention KV entries to the native continuation layout.
- 7: Map the selected hidden-state row to the next drafter input.
- 8: Preserve the native pending-token and position convention.
- 9: **return** Continuation boundary and emitted-token accounting

suffix predictor; this changes the proposal source and must be recorded separately from tree topology [6], [9].

The attention fork applies an additive ancestry bias to speculative suffix scores while leaving the context visible. It retains the native prefill route. Recurrent nodes obtain their parent state from the pre-tree state or node-local scratch. After device selection, accepted-path replay and remapping establish the continuation state [6]. Cache checkpoints and restored rows must obey the same contract; a correct cold first request does not establish correctness after a checkpoint hit or page reuse [7].

B. Accepted state with the wrong attention KV

The archived remap failure left recurrent and convolution state aligned with the selected path, while attention read a neighboring sibling's KV entry after a non-contiguous path was accepted [7]. The fix copies each selected node's KV entry from its verification slot into its linear continuation slot. Its role is state movement rather than a new attention equation. Algorithm 1 makes the missing step explicit.

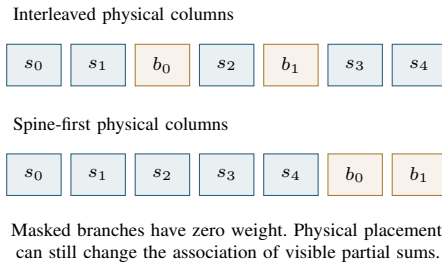


Fig. 3. Illustrative physical slot layouts for one spine query. Blue entries are visible spine keys and amber entries are masked branches. The repair keeps shared-spine positions fixed within the tested execution layout. The drawing is a mechanism illustration, not a new numerical measurement.

The associated token fixture changed from 15/15 failures to 0/15 across the reported eager, graph, and cold-cache checks. In served-script checks, the repaired cat8 and cat6 arms had zero observed corrupted-identifier flags among 84 and 61 scripts respectively [7]. The pre-fix observations came from earlier runs, so their rates should not be treated as a randomized estimate of the repair’s effect. The fixture supplies the more direct causal witness.

A later launcher audit found that previously validated remap and reorder flags were enabled in narrow launchers but disabled in the general campaign launcher. The corrected campaign therefore records post-boot engagement, not merely source-code availability [10]. This is why the identity of a compiled implementation and the effective configuration belong to the evidence boundary.

C. Masked branches can change the shared spine

In exact arithmetic, adding masked keys contributes zero to attention. In the tested FlashAttention-2 route, however, physical score columns also determine the association of the online-softmax reduction [8], [19]. Interleaving extra branch slots changes the positions of surviving spine keys. The visible values are unchanged, but their partial sums need not be associated identically.

The recorded witness shows a one-unit-in-the-last-place discrepancy in a bfloat16 output at magnitude eight: $2^{3-7} = 0.0625$. The diagnostic identifies the shared attention spine and reports a greedy acceptance difference of 0.087 drafts/event between the compared tree variants [8]. Independent stochastic task traces cannot establish the same claim. A meaningful superset check fixes the common prefix, proposals, target reference, and decision rule. Under greedy longest-matching-path selection with unchanged shared-node outputs, adding candidates cannot remove an existing matching path. A numerical change can violate that premise.

The repair places the shared spine contiguously before other tree slots and propagates the permutation through the relevant maps. In the recorded observation harness and in-process check, the reordered shared spine matched the spine-only reference byte-for-byte; an interleaved control retained the discrepancy [8]. This is evidence for the tested route and shape family. Different compiler choices, kernel buckets, quantization

formats, or sampling operations require their own checks. A fixed layout alone is not a universal floating-point identity theorem.

D. Reconstruction, rounding, and accumulated disagreement

The archive includes an earlier local WY prototype. Its investigation reports a recurrent-state error of approximately 1.66×10^{-3} and a subsequent maximum logit discrepancy of 3.32. The diagnosed state-write path reused an excessively rounded output representation. A separate state calculation reduced the isolated error; later assessments retained the algebraic equivalence and left broader acceptance-based validation open [10]. These are documentary observations about our historical prototype, not a reproduction or refutation of TreeWY. The evidence manifest identifies both the failure record and its later correction.

Other archived layer-localization studies examined our sequential tree route as well. They motivate a distinction between a local numerical discrepancy and its effect after further model layers or recurrent steps. A first-order explanatory model is $e_{\ell+1} \approx J_{\ell}e_{\ell} + \delta_{\ell}$: later layers can amplify an existing residual while introducing additional error. This expression is not a fitted error bound. Small local errors, or matching current outputs, do not establish matching future logits [10].

The numerical reference must also be named. The pinned native speculative-update and one-token decode paths differ in gate rounding and state-storage boundaries. Our scan and replay share a sequential update body, but their compiled agreement still requires GPU checks [10]. Thus a real-arithmetic reconstruction identity does not by itself establish deployment agreement; conversely, disagreement with our selected reference does not establish an algebraic defect. E7a and E7b isolate verification arithmetic, state commitment, and subsequent continuation before measuring any deployment benefit. The E7a kernel-level pilot (Section VI-F) measures the first two of these boundaries: the two pinned native paths differ from each other by 2.2×10^{-4} to 1.7×10^{-3} of the state maximum on the tested operands, and an isolated ablation shows the difference is the decode kernel’s documented bfloat16 rounding of the write gate (an oracle applying only that rounding matches it at the fp32 floor).

V. EVALUATION PROTOCOL AND EVIDENCE STATUS

A. Campaigns are separate experimental objects

The main empirical material concerns Qwen3.6-27B-FP8 on GB10/DGX Spark [10], [26]. Table II distinguishes the archived campaigns. They use different implementation stages and, in some cases, different harnesses. They are not replicates of a single final system. The accompanying evidence manifest records file paths and SHA-256 hashes; the accounting script recomputes the H3 table from stored JSON without running inference.

The H3 campaign configures four concurrent requests, enables prefix caching, and runs a 16-task Astropy subset drawn from SWE-bench Verified through qwen-code. Native MTP-5 uses five draft steps. MTP-11 matches the maximum

TABLE II

EVIDENCE CARRIED INTO THIS DRAFT. “VERIFIED ARCHIVE” MEANS THE REPORTED OBSERVATION WAS CHECKED AGAINST AN AVAILABLE SOURCE; IT DOES NOT MEAN THE EXPERIMENT WAS RERUN ON THE CURRENT CODE. P0 AND A KERNEL-LEVEL E7A PILOT ARE EXECUTED; E2/E7B, E1, AND CONDITIONAL E3 REMAIN PLANNED; E7C IS A DEFERRED EXTERNAL COMPARISON.

ID	Evidence	Supported statement	Important boundary
H1	KV-remap report [7]	Fixture repair and observed post-fix script checks	Not a distribution proof or a randomized quality comparison
H2	Spine-layout report [8]	Numerical witness, shared-spine repair, later acceptance values	Byte agreement scoped to tested reference and route
H3	Postfix3 JSON and close-out [10]	Three-arm acceptance and complete decode-step accounting	Historical nominal-B4 campaign; duplicated constraints reconstructed
H4	Paired attention optimization [11]	27.03 ms local step reduction in its configuration	Different stage from H3; not a composed tree-over-chain result
H5	Repeated-task audit [10]	Limits of the 16-task quality sample	Same Astropy subset; heterogeneous historical arms
H6	Archived numerical investigations	Failure mechanism and later correction	Documentary evidence; original captures not reproduced
P0	Configuration audit (Section V)	Sampling reconstruction; no new model measurements	Historical binary gaps retained
E7a	Kernel-level pilot (Section VI-F)	Same-operand arithmetic of three local realizations against a float64 oracle and both pinned native kernels	Dependent historical operand sets and labeled synthetic inputs; no fresh pre-fixes; not the authors’ systems
E2/E7b; E1; E3	Planned	—	Not executed

draft depth of the tail6 tree, whose later suffix uses a different proposal source [9], [10]. It is a useful strong control, but not a pure topology ablation. Task resolution follows the benchmark evaluation; the small chosen subset does not represent the full benchmark [27].

B. Sampling provenance remains an explicit limitation

Historical reports label several campaigns with requested temperature 0.6. The archive documents duplicated sampling constraints [28]. P0 reconstructs the three July patchers at their recorded, distinct source revisions against the launchers’ pinned image: each produces two target-logit constraint calls, while the selected current revision produces one. This is source reconstruction, not recovery of the original patched runtime. The target-row temperature divisors compound to 0.36, but top-k/top-p filtering also repeats and bonus paths differ; the run is not equivalent to uniform temperature-0.36 sampling. We retain the requested settings as *nominal*. Historical loaded-extension and model-file identities remain unresolved; current file hashes cannot establish July identity. The artifact includes the source reconstructions and per-arm audit manifests.

The stored numbers remain observations of the archived runs, useful for describing behavior and diagnosing cost. They do not provide a controlled-temperature estimate of the final system’s advantage. Likewise, an available correctness test counts as a current passing result only when its execution record establishes the relevant route [10].

C. Tokens, events, physical steps, and service time

Let a_e be accepted drafts and c_e be actual emitted tokens in request verification event e . For ordinary nonterminal events, $c_e = a_e + 1$ because of a correction/bonus token. End-of-sequence and truncation must instead use the actual count. A physical GPU step can contain several request events. All cost components must use a common denominator before addition.

Writing bars for quantities normalized to one request event, the decode-step accounting rate is

$$R_{\text{decode}} = \frac{\bar{c}}{\bar{t}_{\text{draft}} + \bar{t}_{\text{verify}} + \bar{t}_{\text{commit}} + \bar{t}_{\text{host}}}. \quad (4)$$

The H3 wall instrument uses start-to-start intervals between consecutive pure-decode steps, caps idle intervals, and breaks the chain on mixed or prefill steps. Thus it includes decode-step host work but is *not* total generated output divided by end-to-end service time [10]. Time to first token (TTFT), time per output token (TPOT), queueing, cache misses, and campaign wall time are separate quantities.

An earlier analysis combined draft/commit timings per physical step with verification timings per request event. The corrected raw files divide the former by measured events/step before summation. That is an accounting repair, not a kernel speedup. The resulting GPU-only-rate/wall-rate ratios are 1.017–1.025; a ratio slightly above one is consistent with excluded host cost. The older approximately 0.53 mismatch is superseded [10].

VI. RESULTS

A. State and layout findings

Table III summarizes H1 and H2. The repaired state mapping eliminates the fixture’s observed failure, while the layout investigation identifies a separate numerical cause. These results address distinct failure modes. A system can satisfy the recurrent-path equation yet violate attention-KV publication, and can satisfy an abstract mask yet change the finite-precision output [7], [8].

After the layout repair, the recorded B4 cat8–cat6 acceptance difference is 0.1664 drafts/event. This is consistent with the restored opportunity of the wider tree, but it is not a paired estimate of the greedy witness’s effect: the deployed arms

TABLE III

ARCHIVED CORRECTNESS AND ACCEPTANCE OBSERVATIONS. COUNTS HAVE DIFFERENT UNITS; THEY ARE NOT POOLED INTO ONE FAILURE RATE.

Observation	Recorded result
KV-remap token fixture	15/15 failures to 0/15
Post-fix cat8 script flags	0/84 corrupted identifiers
Post-fix cat6 script flags	0/61 corrupted identifiers
Reordered shared spine	Byte match in tested checks
Later B4 cat8 acceptance	3.4996 drafts/event
Later B4 cat6 acceptance	3.3332 drafts/event

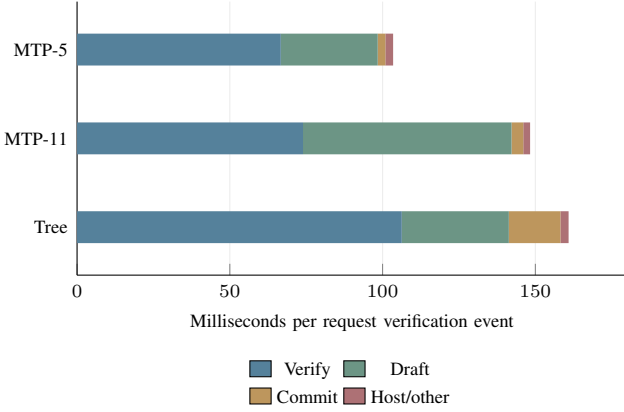


Fig. 4. H3 cost components on a common per-request-event basis. The source records milliseconds per physical step for draft and commit; these are divided by the measured request events/step. Different campaigns’ component estimates are not combined.

follow different stochastic trajectories [8]. Equal task counts in that campaign likewise cannot establish equivalence. A direct structural claim rests on the matched-input numerical check.

B. The later tree loses to both chain controls

Table IV reports H3. The tree commits about 19.5% more tokens/event than native MTP-5, but costs about 55.5% more per event. The larger numerator does not compensate for the denominator. MTP-11 commits the most tokens per event of the three, yet its longer drafting path makes it slower than MTP-5 on the same decode-step measure [9], [10].

Figure 4 shows where the difference arises. Verification costs 106.28 ms/event for the tree and 66.68 ms/event for MTP-5; commitment costs 16.89 versus 2.49 ms/event. Drafting is 35.04 versus 31.72 ms/event. The MTP-11 draft cost is 68.26 ms/event [10]. These identify expensive operations in this route. They do not establish that another representation must pay the same replay or verification cost.

Configured batch size is not realized occupancy. The measured request events per pure-decode step range from 2.74 to 2.80 in this campaign. Larger nominal capacity cannot be substituted for the observed denominator. The recorded task totals are also descriptive: one run per arm on the same selected subset does not support a significance claim or a quality ranking [9], [10].

C. A conditional break-even calculation

At fixed comparison conditions, Eq. (4) gives the acceptance threshold

$$\bar{a}_{\text{tree}}^* = (\bar{a}_{\text{native}} + 1) \frac{\bar{t}_{\text{tree}}}{\bar{t}_{\text{native}}} - 1. \quad (5)$$

Using the unrounded H3 values yields 5.88 accepted drafts/event for parity with MTP-5. Alternatively, at the observed tree acceptance, its event-normalized time would need to fall to 123.66 ms, a 23.1% reduction. The implementation of this calculation in the paper artifact corrects a historical note that omitted the bonus token [10].

This is a local accounting threshold. Increasing depth, changing the drafter, changing batch occupancy, or reducing cache capacity can change both numerator and denominator. It is not a forecast that the tree can reach the threshold at unchanged cost, nor a claim that no alternative tree algorithm can do better.

D. A reliable local optimization, with composition still open

H4 examines attention work partitioning at a later implementation stage. Pairing query-head work uses previously idle query-tile capacity and reduces the number of thread blocks in the tested B4 route [11]. Its paired timing summary reports a mean reduction of 27.03 ms/step across four passes, with standard deviation 13.77 ms and a reported one-sided 95% lower bound of 10.82 ms. The summarized baseline and optimized times are 408.313 and 381.284 ms. These are physical-step values for that campaign; subtracting 27.03 ms from H3’s event-normalized time would mix experiments and units.

The later archive also contains a provisional single-pass GDN result and correctness work on a batched tree-accept walk, but not the completed composed comparison [10], [11]. Table V deliberately preserves those statuses. Kernel interactions, changed bottlenecks, and workload variation make addition of individual best deltas unjustified.

E. Quality and the original B1 observation

The original manuscript reported 23.88 versus 18.80 token-weighted request-decode tokens/s for a small June B1 gate, with 18.51 versus 17.80 under a per-request-equal view [6]. Those are different metrics and a different campaign from H3. We retain them as historical context, subject to the sampling audit, rather than as the current revision’s acceleration headline. Its recurrent-oracle rescore sample was not a distribution-distance proof.

H5 summarizes 21 historical arms on the same 16 Astropy tasks: seven tasks always resolved, four never resolved, and five varied [10]. Implementations differ across arms, so these cannot be pooled as 21 independent replications of one system. The sample is useful for detecting gross regressions; it is weak evidence for non-inferiority or general task quality. An optional newer quantization campaign also records external retrieval of a benchmark gold patch, requiring provenance review before its affected outcomes can be used. No such outcome supports the core conclusions here.

TABLE IV

H3: HISTORICAL NOMINAL-B4 CAMPAIGN, PREFIX CACHE ENABLED, 16 ASTROPY TASKS. VALUES ARE RECOMPUTED FROM THE CORRECTED RAW JSON. NOMINAL TEMPERATURE WAS 0.6; EFFECTIVE SAMPLING PROVENANCE REMAINS UNDER AUDIT. TOKENS/S HERE IS EQ. (4), NOT FULL-SERVICE THROUGHPUT.

Arm	Accepted/event	Committed/event	ms/event	Decode tokens/s	Events/step	Resolved
Native MTP-5	3.422	4.422	103.45	42.74	2.79	10/16
Native MTP-11	4.923	5.923	148.28	39.95	2.74	8/16
Tree, tail6	4.286	5.286	160.88	32.85	2.80	10/16

TABLE V

LATER OPTIMIZATION EVIDENCE. PROVISIONAL AND UNMEASURED ITEMS ARE EXCLUDED FROM FACTUAL COMBINED-SPEED CLAIMS.

Change	Evidence status
B4 query-head pairing	Verified archive: 27.03 ms paired mean reduction
B4 single-pass GDN scan	Provisional 8.984 ms observation; final paired validation pending
Batched tree-accept walk	Correctness checks archived; deployment timing pending
Combined final stack vs. chains	Planned E1/E3; no measured conclusion

F. E7a pilot: three arithmetic realizations on identical operands

After P0 we executed the E7a comparison at the kernel level on 21 September 2026 (artifacts of record, with hashed sources, compiled-kernel hashes, integer-view equality metrics, and telemetry, under `experiments/out-20260921T230026Z-e7a-ladder-v3/`; the earlier same-operand run `out-20260921T222616Z-e7a-ladder/` is retained). Inputs are the rounded operands of one GDN layer step: bfloat16 q, k, v , bfloat16 raw gates, fp32 $A_{\log}$, bfloat16 gate bias, the fp32 pre-tree state, and the tree descriptor. Four operand sets are historical June-2026 captures at layers 62, 1, 12, and 0 of the served ten-node caterpillar (depth 5, B1). They are dependent operand sets whose independent-prefix count cannot be recovered from the files; they are not fresh prefixes and contribute nothing to the planned eight- and 32-prefix sets. Five further sets are labeled synthetic with historical-like gate statistics: chains of depth 1, 5, and 11, a binary tree of depth 3, and the caterpillar; a synthetic four-request batch exercises the batched kernels. Three references are kept apart: a float64 serial recurrence on the same rounded operands, the pinned native speculative-update kernel run per root-to-node chain, and the pinned one-token decode kernel run token by token.

Mechanism A is this worktree’s production scan and accepted-path replay. Mechanisms B and C are our Triton reimplementations of the two published families on the same masked system $(I + G)U = R$: forward substitution (the WY/TreeWY family) and the finite Neumann series with d terms (the Bole family), each followed by the compact reconstruction $S_a = P_a S_0 + \sum_{j \prec_a} (P_a/P_j) u_j k_j^\top$. Dense products run either in fp32 or in tf32 tensor-core precision; the corrected June-2026 WY kernel is included as a historical realization. Every accepted prefix, including zero acceptance, is committed, first from each method’s own factors and then

from identical oracle factors.

Table VI summarizes. In float64 the three mechanisms and the oracle agree to about 10^{-15} . With fp32 products every realization sits at the fp32 floor against the oracle on all nine sets: verifier outputs within $0.9\text{--}5.2 \times 10^{-7}$ of the reference maximum, correction factors within $1.7\text{--}3.4 \times 10^{-7}$, committed states within $1.0\text{--}3.7 \times 10^{-7}$. Commit from identical oracle factors is at the same floor, so on these operands the compact reconstruction adds no error beyond fp32 roundoff. In a diagnostic comparison with fp32 output stores (a separately compiled specialization of both kernels, not the deployed bfloat16 path), the production scan’s outputs are bit-identical to the native speculative-update kernel’s on every element of all nine sets (integer-view equality); the deployed bfloat16-store outputs are bit-identical in 99.998–100% of elements (two sets each differ in a single element by one unit in the last place), and the bfloat16 outputs of B and C are bit-identical to native’s in 99.97–99.99% of elements. With tf32 products the same kernels lose three orders of magnitude (outputs $6.5 \times 10^{-4}\text{--}1.4 \times 10^{-3}$; bfloat16 agreement 74–79%). The precision of the dense products, which the Bole paper does not state, decides whether a reconstruction route can meet a native reference.

The two pinned native paths disagree with each other by $2.2 \times 10^{-4}\text{--}1.7 \times 10^{-3}$ of the state maximum, with 75–94% bfloat16-output agreement. An isolated oracle-convention ablation attributes the gap: a float64 oracle that applies only the decode kernel’s documented bfloat16 rounding of the write gate matches it at $1.2\text{--}4.0 \times 10^{-7}$, whereas an oracle that applies only its $x/\sqrt{\cdot}$ normalization leaves the full gap ($2.2 \times 10^{-4}\text{--}1.7 \times 10^{-3}$) in place; in float64 the normalization convention moves the state by at most 5×10^{-15} (this identifies which declared convention the fp32 kernel follows; it does not measure the two fp32 kernels’ entire implementation difference). On these operands the gap is therefore the declared gate rounding, not an implementation error, and for the outputs where the two paths differ no single result can lie within the fp32 floor of both. Controls: sibling reordering is bitwise-invariant for A on all sets and for B and C on eight of nine (last-bit differences on the bushy binary tree from changed accumulation order); changing the padded node count changes last bits ($\leq 1.2 \times 10^{-7}$ absolute); dropping one Neumann term fails (factor error up to 0.29), so the nilpotency term is load-bearing; for all four requests of a synthetic batch, the batched outputs, factors, and commits are bit-identical (integer view) to the single-request results. Along the depth-11 chain the committed-state error grows from 5×10^{-7} to $1.0\text{--}1.3 \times 10^{-6}$ absolute,

TABLE VI

E7A PILOT ACROSS NINE OPERAND SETS (FOUR HISTORICAL, FIVE SYNTHETIC). ERRORS ARE MAXIMA RELATIVE TO THE REFERENCE MAXIMUM, GIVEN AS THE RANGE OVER SETS; “FP32” AND “TF32” DENOTE THE PRECISION OF THE DENSE PRODUCTS IN B AND C. COMMIT ROWS COMPARE THE COMMITTED STATE FOR EVERY ACCEPTED PREFIX. BF16 AGREEMENT IS THE FRACTION OF DEPLOYED-WIDTH BFLOAT16 OUTPUT ELEMENTS BIT-IDENTICAL (INTEGER VIEW) TO THE NATIVE SPECULATIVE-UPDATE KERNEL’S. LOCAL REIMPLEMENTATIONS ONLY; NOT THE AUTHORS’ SYSTEMS.

Realization	Verify out	Commit (own)	Commit (oracle factors)	bf16 agreement
A scan + replay (production)	0.9–3.1e-7	1.0–3.7e-7	—	0.99998–1
B forward subst. fp32	2.8–5.2e-7	1.0–2.8e-7	1.0–2.1e-7	0.9997–0.9999
C finite Neumann fp32	2.8–5.2e-7	1.0–2.8e-7	1.0–2.1e-7	0.9997–0.9999
B forward subst. tf32	6.9e-4–1.4e-3	3.0–8.1e-4	2.6–8.9e-4	0.744–0.786
C finite Neumann tf32	6.5e-4–1.4e-3	3.0–8.1e-4	2.6–8.9e-4	0.745–0.788
June-2026 WY kernel (fp32 closed form)	1.4–2.1e-7	0.9–2.2e-7 (materialized)	—	—
Native speculative update (reference)	= A	1.0–3.7e-7	—	1
Native one-token decode vs. oracle	—	2.2e-4–1.7e-3	—	0.751–0.937
Native one-token decode vs. its own seam oracle	—	1.2–4.0e-7	—	—

still about 10^{-7} relative. A separate synthetic tiny-gate pass (out-20260921T231030Z-e7a-tinygates-v3/; chains of depth 11 and 5 and the caterpillar with cumulative log-decay down to -912 , so that the path decay P_i underflows to exactly zero in fp32) produced no non-finite value in any realization: outputs and committed states stayed within $0.9\text{--}5.8 \times 10^{-7}$ of the oracle because every implementation evaluates path-decay ratios as $\exp(\log P_i - \log P_j)$ with the exponent masked before exponentiation, never as a literal quotient of underflowed products. The same run shows that with vanishing gates the strict-ancestor coupling vanishes too, so even the truncated Neumann series is accurate there; the truncation control fails only when coupling is present.

Cost is reported as kernel timing only, for one layer at B1, from the v3 run, whose 1 Hz compute-process inventory sampled no process other than our container and whose in-band probe drifted by at most 4.8% across timing blocks (2.7% in the tiny-gate run); rows perform different work and are not speedups. On the four historical sets (CUDA-event medians): the production scan takes $89 \mu\text{s}$ with its deployed bfloat16 output store and no state export; the B and C verifiers take $107\text{--}108$ and $101\text{--}103 \mu\text{s}$ with matched bfloat16 output stores ($109\text{--}111$ and $101 \mu\text{s}$ with fp32 stores), both also writing their factors, and $41\text{--}42$ and $31\text{--}32 \mu\text{s}$ with tf32 products; the replay commit of a depth-5 path writes six state rows in $102\text{--}105 \mu\text{s}$, while the compact commit writes one row in $18\text{--}19 \mu\text{s}$ ($54 \mu\text{s}$ for a zero-accept replay); the native speculative-update kernel run on the depth-5 chain with preallocated inputs takes $113\text{--}121 \mu\text{s}$ (its wrapper still allocates its outputs), and the native one-token decode kernel takes $24\text{--}25 \mu\text{s}$ per token. Across the tiny-gate sets, the sequential scan grows with node count ($59 \mu\text{s}$ at six nodes, 90 at ten, 134 at twelve) while the B and C verifiers stay at $106\text{--}107$ and $95\text{--}99 \mu\text{s}$ because their padded system is fixed. Transient memory per request and layer is 396 KB of factors for the compact route, $(d+1) \times 3.1 \text{ MB}$ of committed rows for replay, and 50 MB for any route that materializes every node state. This pilot does not show

model-level propagation, sampler acceptance, or continuation (E2/E7b), any fresh prefix, any serving effect (E1), or anything about the authors’ implementations.

VII. LIMITATIONS AND PLANNED VALIDATION

The measured conclusions apply to archived implementation stages, one main checkpoint family, and a limited task subset. The state contract is more general than its test coverage. Native-reference numerical agreement, sampler-law correctness, and deployment quality must remain separate claims. Instrumentation also has limits: the pure-decode wall metric intentionally excludes mixed/prefill intervals and cannot establish full-service throughput by itself.

Table VII lists the selected validation scope and deferred extensions. After configuration reconciliation (P0), three core experiments compare arithmetic on identical inputs (E7a), test numerical propagation and accepted-prefix continuation (E2/E7b), and measure the frozen implementation against strong chains (E1). E3 is conditional on promoting unfinished optimizations. Except for the E7a kernel-level pilot reported in Section VI-F, these are planned experiments; no other new measured results are implied.

E7a compares local reimplementations of published mechanisms, not the authors’ systems. Identical captured operands and chosen paths isolate solver error from commit error. High-precision recurrence and each pinned native GPU path serve distinct reference roles. Its kernel-level pilot is reported in Section VI-F; the fresh eight-prefix pilot and the 32-prefix confirmation set have not been collected. E7b holds teacher-forced inputs fixed while replacing verification, commitment, and their combination separately; only then does it test actual acceptance and continuation. An eight-prefix diagnostic pilot sets the analysis and costs before a separate 32-prefix confirmation set is frozen. These are bounded diagnostic samples, not a powered task-quality study. Errors beyond the declared criteria preclude deployment promotion but remain reportable numerical findings.

TABLE VII

FINAL RECOMMENDED SCOPE. CORE EXPERIMENTS ARE E7A, E2/E7B, AND E1, PRECEDED BY P0; E3 IS CONDITIONAL. THE REMAINING ENTRIES ARE DEFERRED. E7A HAS AN EXECUTED KERNEL-LEVEL PILOT (SECTION VI-F); THE REMAINING NEW MEASUREMENTS ARE PLANNED, NOT EXECUTED.

ID	Question	Minimum comparison	Claim enabled
P0	What actually ran?	Archived binary, flags, sampling and denominator reconciliation	Correct configuration labels
E1	Does the final tree beat strong chains?	Tree, MTP-5, depth-matched chain; B1/B4; paired independent boots	Current-stack performance
E7a	Do arithmetic realizations agree?	Same operands: sequential, triangular solve, finite-Neumann; output and commit isolated	Numerical and local-cost characterization
E2/E7b	Do errors amplify or corrupt continuation?	Fixed-prefix stage substitutions; layer/final logits; state/KV/conv, sampler and cache fixtures	Scoped continuation and implementation agreement
E3	Do optimizations combine?	Conditional scan/accept-walk toggles, individually and together	Composed local speedup
E4	How does memory affect service latency?	Deferred: identical arrival trace and cache-capacity controls	Serving and cache explanation
E5	Is broader task quality preserved?	Deferred: held-out tasks and powered quality design; exact sampler checks stay in E2	Broader quality evidence
E6	Does the result extend to NVFP4?	Deferred: matched native control and repeated composed tests	Quantization extension
E7c	How do authors' systems compare?	Deferred: feasible matched external implementations	Competitive system comparison

E1 uses controlled prefix replay rather than conflating it with fixed-arrival serving or live agent execution. Three arms, two actual batch sizes, and three independent paired boots give an initial 18 timing cells. If a reconstruction route passes E2/E7b and is integrated, retain the sequential tree as an ablation and add that route as a fourth arm, yielding 24 cells. Replication is set from pilot variance and a prespecified practical precision target; the final test is not repeated until a favorable result appears.

E2 should map existing coverage before adding tests. The repository already includes an analytic multidraft sampler identity, multistep sampling tests, a biased-selector negative control, and accepted-path state fixtures [10]. Missing coverage concerns their connection to the current deployed GPU route, including zero/full acceptance, sibling paths, page reuse, and pending-token continuation. Source availability alone is not a passing execution record. No proof of full distribution preservation is inferred from the historical numerical or garble checks.

VIII. CONCLUSION

GDN Tree-Scan studies the implementation boundary between tree verification and the next native serving iteration. A correct continuation requires agreement among recurrent state, convolution history, attention KV, physical row mappings, and the sampler. The archived failures show that satisfying one surface can leave another inconsistent.

The KV-remap fixture improved from 15/15 failures to 0/15, and the numerical investigation produced a shared-spine repair with byte agreement in its tested checks (Table III). In the later three-arm campaign, however, the tree's 5.286 committed tokens/event did not repay its 160.88 ms event cost: its decode-step rate was 32.85 versus native MTP-5's 42.74 tokens/s (Table IV). A separate paired attention optimization reduced its tested B4 step by 27.03 ms (Table V) [10].

These findings support a scoped account of state integrity, numerical execution, and serving cost. They do not establish superiority over direct GDN tree methods or a

distribution-preservation guarantee for the current implementation. Effective-configuration reconciliation, matched current-stack timing, and composed correctness checks are the next validation steps.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving mamba2 with delta rule," 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state-space models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, "GDN Tree-Scan consolidated implementation and measurement receipts," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/55f55854328b37f262e97d57b5863d8fadd7ff76, 2026, commit-pinned repository receipts for deploy TPS, equivalence gates, runtime, launchers, kernels, and configuration.
- [7] —, "Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native," https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] —, "One Bit in the Last Place: Why a Wider Speculative Tree Accepted Fewer Tokens," https://macoredroid.github.io/Lumo_FlyWheel/spine-reorder.html, 2026, author experiment archive; accessed 2026-09-21.
- [9] —, "Keep or Replay: Why a Speculative Tree That Accepts More Still Loses to the Linear Chain," https://macoredroid.github.io/Lumo_FlyWheel/keep-or-replay.html, 2026, author experiment archive; accessed 2026-09-21.
- [10] —, "Lumo FlyWheel implementation and experiment archive," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c, 2026, revision evidence snapshot; individual campaigns retain their own run identities.
- [11] —, "Twenty-Seven Milliseconds, Shipped," https://macoredroid.github.io/Lumo_FlyWheel/numbers-that-didnt-survive.html, 2026, author experiment archive; accessed 2026-09-21.
- [12] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, robust, and hardware-aware speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>

- [13] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-2: Faster inference of language models with dynamic draft trees," 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [14] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "Opt-tree: Speculative decoding with adaptive draft tree structure," 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [16] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [17] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention," <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [18] SGLang Project, "Qwen3-Next SGLang documentation," <https://lmsys.org/mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [19] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [20] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [21] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "Specmamba: Accelerating mamba inference on fpga with speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [22] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [23] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, "Bole: Efficient Tree Speculation for Hybrid-Attention Language Models," 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [24] S. M. Ghantasala, "TreeWY: Speculative Verification for Gated DeltaNet Hybrids," 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [25] Dao AI Lab and NVIDIA, "ReplaySSM: Cache SSM Inputs Instead of State," <https://github.com/vllm-project/vllm/issues/47572>, 2026, implementation RFC; accessed 2026-09-21.
- [26] Qwen Team, "Qwen3.6-27B-FP8," <https://huggingface.co/Qwen/Qwen3.6-27B-FP8>, 2026, public Hugging Face model page; accessed 2026-09-21.
- [27] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [28] Z. Ma, "Every Lever We Pulled: A Speculative-Decoding Speed Campaign in Full," https://macoredroid.github.io/Lumo_FlyWheel/every-lever.html, 2026, author experiment archive; accessed 2026-09-21.